LLM Benchmark

Nome del Relatore

June 23, 2026

Contents

Introduction

Goal and Keypoints

Goal: create a reliable, scalable and user-friendly benchmark to test the planning capabilities of LLMs on planning problems.

Keypoints

- Problems Classification
- Hardware Platform and LLM Models Selected
- Outputs and Evaluation Metrics

General Structure

This slide is used to show the structure of the repo, under the root folder we have:

- **analysis**: contains the material needed to write reports and the report itself.
- **archive**: contains material provided by the professor and a PDDL guide written to help understand key features of the PDDL.
- **Benchmark Framework**: main folder containing the work done to create this benchmark.

```
LLM Benchmark/
├ analysis/
│ ├ domain complexity/
│ ├ notebooks/
│ └ Reports/
├ archive/
│ ├ Papers/
│ ├ Planning Domains/
│ ├ References/
│ └ Quick PDDL Guide/
├ Benchmark Framework/
│ └ ...
└ README.md
```

General Structure II

Now the Benchmark Framework folder is highlighted, note that to simplify not all the sub folders are shown.

- *evaluators*: code for running evaluation and parsing of oututs
- *Leonardo script*: files to gain access to leonardo booster
- *outputs*: store outputs of benchmark
- *prompts*: prompts written to be feed to LLMs
- *runner*: contains code to run some tests with LLMs on the booster
- *scripts*: contains code necessary for the scoring of the inputs
- *tasks*: problems selected
- *utils*: executable of validator
- *results*: heatmaps showing output metric results
- *run benchmark*: file that runs whole working sequence

```
Benchmark Framework/
├ evaluators/
├ Leonardo script/
├ outputs/
├ prompts/
├ runner/
├ scripts/
├ tasks/
├ utils/
├ results/
└ run benchmark.py
```

Problems' Classification

Classification Purpose

Purpose: In order to obtain reliable outputs to be evaluated after we for sure need a way to classify the PDDL problems to be fed to the LLMs chosen, since we had available 20 different problems a easy but effective way to classify them was needed to choose wisely the problems to be used.
The main idea behind this Input Classification develops around two main concepts:

- need of a numerical score representy the difficulty of the problem.
- classify problems based on the type of reasoning to be employed to solve the problem.

Implementation: Numerical Score

As stated before the idea of using a numerical score to classify the difficulty of the problems came up because a relatively simple and repeatable way to evaluate problems was needed; in order to obtain the score the code in *LLM benchmark/scripts/score domains complexity.py* has been designed. The main idea behind the computation of the score is to evaluate for each instance of a PDDL problem the presence or the absence of features that we believed could increment or decrement the difficuly of a problem, specifically:

- N. of Objects
- N of possible Actions
- N. of Goals
-

A raw score for each instance of a problem is therefore computed then for each domain a summary is obtained.

Implementation: Numerical Score II

Implementation: Reasoning Type

Hardware platform and Models selected

Hardware Platform and Models Selected

DA RIEMPIRE

Hardware Platform and Models Selected II

DA RIEMPIRE

Output and Evaluation Metrics

Output Structure

DA RIEMPIRE

Evaluation Metrics

DA RIEMPIRE

Conclusion

Conclusion

DA RIEMPIRE